IPD432: Diseño Avanzado de Sistemas Digitales
Tarea 2 - 2026

1. Indicaciones generales

- La tarea se puede realizar en forma individual o en grupos de dos personas.
- Debe entregar un informe escrito en formato IEEE conference de doble columna, utilizando los templates disponibles en el siguiente link: `https://www.ieee.org/conferences_events/conferences/publishing/templates.html` (se recomienda el uso de LaTeX).
- Si lo desea, puede entregar su informe en inglés (opcional).
- Habrá penalización en la nota por detalles de formato y presentación (faltas de ortografía, errores gramaticales graves, texto desalineado, figuras en mala calidad, etc.). Esto quedará a criterio del profesor, considerando los comentarios entregados en la revisión de tareas anteriores. Sea prolijo, ya que la evaluación del informe se realizará con el mismo criterio que se evalúa un artículo científico. **Se permite, y recomienda, el uso responsable de herramientas de Large Language Models (LLMs) para mejorar la presentación de su informe escrito.**
- La fecha límite de entrega para la tarea (códigos e informe) es el 14 de octubre de 2026 a las 11:59 hrs. al correo *gcarvajalb@gmail.com*.

2. Objetivos

Por medio de esta tarea, se espera que se familiaricen y ganen experiencia práctica en los siguientes conceptos:

- Fundamentos y terminología para la integración de coprocesadores y aceleradores.
- Diseño de datapaths y control paths.
- Uso de Intellectual Property (IP) cores.
- Reportes de uso de recursos, análisis de timing estático, y caracterización de desempeño.
- Uso de herramientas para caracterización y depuración del hardware en tiempo de ejecución.

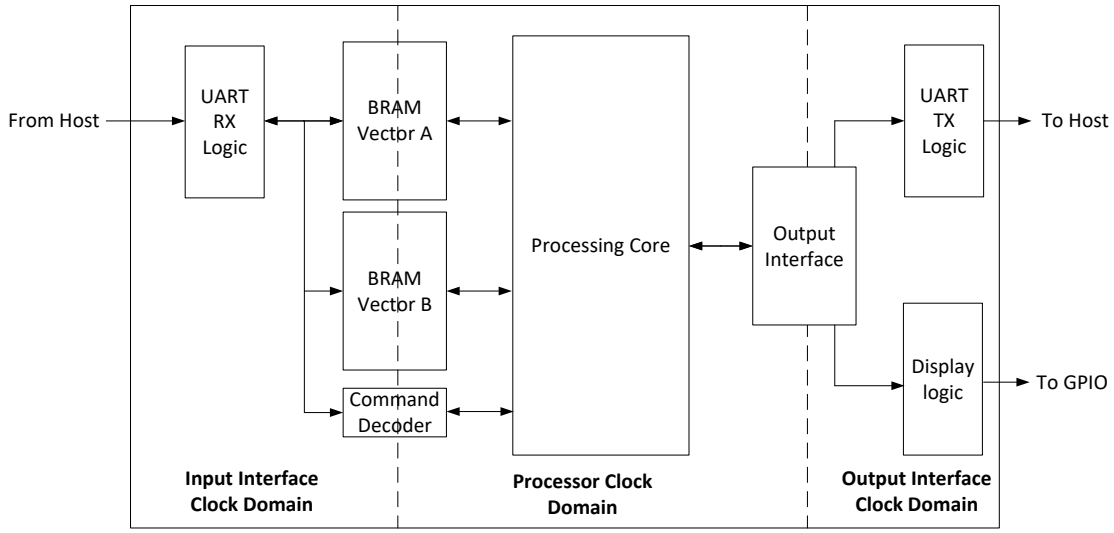


Figura 1: Diagrama genérico de alto nivel de procesamiento en el coprocesador (device). Puede adaptar la estructura a su conveniencia.

3. Actividad de diseño e implementación de hardware

El objetivo de esta actividad es la implementación de un coprocesador especializado en la FPGA que entregue soporte a un procesador principal para realizar operaciones simples sobre vectores. Siguiendo la nomenclatura utilizada y aceptada en el contexto de GPUs [1], nos referiremos al procesador principal como *host* y al coprocesador como *device*[2]. En el caso de esta tarea, el *host* corresponderá a un computador de escritorio y el *device* a un coprocesador implementado en la tarjeta de desarrollo. Ambas unidades se comunican por medio de una interfaz UART.

Para el diseño del *device* considere como referencia el diagrama de alto nivel de la Figura 1. El diagrama contiene dos bloques de memoria del tipo BRAM, cada uno capaz de almacenar 1024 palabras de 11 bits cada una. Estos bloques de memoria almacenan dos vectores A y B, cuyos elementos son cargados desde el host por medio de la interfaz UART. Los bloques de memoria utilizan una configuración *True Dual Port*, lo cual permite separar, en caso de ser necesario, los dominios de reloj para la lógica de escritura/lectura de datos de la lógica de procesamiento. El bloque *Processing Core* permite realizar distintas operaciones sobre los vectores ya almacenados en los bloques de memoria, enviando el resultado de estas operaciones al host y también a puertos GPIO conectados a periféricos de la tarjeta de desarrollo. Las operaciones a realizar sobre los vectores son configuradas e iniciadas desde el host por medio de comandos enviados a través de la UART, los que son decodificados en el bloque *Command Decoder* para iniciar las acciones correspondientes.

Considere los siguientes requerimientos para la implementación del device:

- Apuntando a la usabilidad de su coprocesador, debe diseñar la arquitectura del dispositivo para que pueda ser usado utilizando el template en el script de MATLAB

[1] `https://developer.nvidia.com/blog/easy-introduction-cuda-c-and-c/`

[2] Se sugiere revisar los conceptos de coprocesadores y/o aceleradores. Encontrarán mucha más literatura asociada al contexto de GPUs, pero los conceptos fundamentales son generales y aplican a cualquier tipo de acelerador, incluyendo los basados en FPGA. En las siguientes tareas se explorarán estos conceptos más en detalle.

`coprocessorTesting.m`, entregado junto al enunciado de esta tarea. Este script genera vectores de prueba aleatorios y verifica el resultado con la versión de software.

- Para el envío de vectores desde el host al device, considere que los vectores están almacenados en un archivo de texto con el siguiente formato:

```
Element0
Element1
.
.
.
Element1023
```

  Por simplicidad, asuma que los elementos se interpretaran como enteros sin signo.

- La interfaz de usuario para el coprocesador debe proveer la función `write2dev('file', 'BRAMX', COM)`, donde `file` es el archivo de texto que contiene el vector a almacenar, `BRAMX` es el bloque de memoria donde se quiere almacenar el vector (BRAMA o BRAMB), y `COM` corresponde al puerto a utilizar para comunicarse por la UART. Esta función debe automatizar el llenado de las memorias, leyendo los valores desde el archivo de texto indicado y enviándolos como un solo stream. Por simplicidad, asuma que el usuario proveerá archivos de texto que contengan como máximo 1024 elementos y sin errores [3]. El coprocesador debe incluir lógica para asignar los valores adecuados a las señales de control internas de lectura/escritura en los bloques correspondientes cuando se ejecute esta función.

- La interfaz de usuario también debe proveer la función `command2dev('xxx', COM)`, donde `xxx` especifica una operación a realizar sobre los vectores previamente almacenados. Esta función envía secuencias de bits por la UART que son interpretadas por el bloque *Command Decoder*, el cual configura el *Processing Core*, gatilla el cálculo de la operación especificada, y envía el resultado hacia el host utilizando la UART. Se debe soportar, como mínimo, las siguientes operaciones (puede agregar más si lo desea):

  - `ReadVec`: lee el vector almacenado en el bloque de memoria correspondiente (indicado en otro argumento).
  - `SumVec`: calcula la suma elemento a elemento de los vectores A y B previamente almacenados.
  - `AvgVec`: calcula el promedio elemento a elemento de los vectores A y B previamente almacenados.
  - `ManDist`: calcula la distancia de Manhattan entre los vectores A y B previamente almacenados.
  - `EucDist`: calcula la distancia euclideana entre los vectores A y B previamente almacenados.
  - `DotProd`: calcula el producto punto entre los vectores A y B previamente almacenados.

[3] Esto es el escenario ideal para simplificar la tarea. En la práctica, deberían considerar el caso en que el archivo esté incorrecto, y decidir entre entre realizar una verificación por software para asegurarse que el archivo está correcto antes de enviar los datos o bien incorporar un chequeo en hardware para abortar y evitar estados inesperados en el caso de que algún dato esté corrupto. ¿Cuales serían las ventajas y desventajas de cada caso?

Todas las operaciones operan sobre exactamente 1024 elementos.

- En el caso de que las operaciones realizadas sean la distancia Euclideana, distancia de Manhattan, o el producto punto, debe dimensionar el número de bits del resultado para que alcance todo el rango de operación. Además, en el caso de las operaciones mencionadas, el resultado de la operación debe quedar también registrado en los displays de 7 segmentos de la tarjeta de desarrollo en formato decimal, manteniéndose visible hasta que se realice otra operación. En el caso de realizarse operaciones que no sean de distancia o producto punto, los displays deben quedar apagados o bien mostrar alguna información útil asociada a la última operación realizada. Para todas las operaciones, puede decidir las políticas de truncamiento o redondeo que considere necesaria, reportándolas como decisión de diseño y justificando los efectos esperados.

- La descripción e implementación del bloque `Output Interface` se deja a criterio del diseñador. Por ejemplo, puede optar por usar un BRAM adicional, una máquina de estados para coordinar el envío de datos a la UART a medida que se generan, etc.

- El device implementado en la FPGA debe ser completamente controlado desde el host, no debe requerir manipulación física del usuario (exceptuando el encendido), y una vez encendido debe estar continuamente disponible para recibir comandos (por ejemplo, no debería requerir reiniciarse o presionar un botón luego de cada operación). Por simplicidad, asuma que el usuario no enviará un nuevo comando si no se ha terminado de ejecutar el anterior. Por ejemplo, puede dejar bloqueada la UART si hay una operación en curso para evitar conflictos. Esto simplifica el diseño al omitir el manejo de colas, arbitraje, y/o interrupciones[4].

La revisión y validación de su diseño se hará en base al template provisto en el script `coprocessorTesting.m`. Este script representa la capa de usuario, y puede añadir todos los scripts o funciones necesarias para implementar la funcionalidad requerida. Se enfatiza que el usuario final debe ver la interfaz lo más simple posible y poder usar el hardware en forma intuitiva, siendo responsabilidad del desarrollador ocultar toda complejidad asociada al diseño y manejo del hardware a bajo nivel por medio de librerías que oculten todos los detalles en la codificación de los comandos (por ejemplo, no debe pedir al usuario que recuerde o configure bits usados para codificación de comandos). En otras palabras, el usuario del script solo verá los resultados en la interfaz de usuario, sin siquiera necesitar saber que el hardware externo existe. Una vez finalizado un test, el usuario debe ser capaz de correr otro inmediatamente, sin necesidad de manipular el hardware (cuide la correcta apertura y cierre de puertos y archivos en el script de MATLAB). Los argumentos propuestos para cada uno de los comandos son referenciales, y puede incluir argumentos adicionales si los considera necesarios. Sin embargo, debe reportar toda decisión de diseño en este aspecto y dejar explícita cualquier instrucción para que el usuario pueda usar su código. La usabilidad de su sistema será considerada en la evaluación[5].

Para su trabajo, considere las siguientes etapas:

- Como siempre, parta realizando un diseño de alto nivel para identificar los módulos principales, e implemente su diseño aplicando los procesos que ya conoce para implementación y

[4]Sin embargo, se recomienda como ejercicio pensar en hacer su sistema robusto ante posibles violaciones de estas condiciones o para maximizar la utilización del hardware

[5]No todo es desempeño. Pueden tener el sistema más eficiente del mundo, pero se vuelve inútil si no es usable.

verificación en hardware. Recuerde que este es un proceso iterativo, y en ocasiones las observaciones derivadas de la implementación pueden llevar a modificaciones en el diagrama de alto nivel.

- Utilice un IP de Clocking Wizard para generar el reloj base del sistema. En primera instancia, configure el IP para que tanto el reloj de entrada como el de salida sean de 100 MHz. Toda la lógica debe estar sincronizada por medio del reloj de salida del clocking wizard.

- Trabaje en forma incremental, asegurándose que puede implementar las funciones más simples, verificar su funcionamiento, y luego ir agregando funcionalidad adicional. Por ejemplo, puede partir verificando que es capaz de escribir y leer los datos de la memoria, luego verificar que puede realizar la suma, luego el promedio, luego la distancia de Manhattan, etc. En cada etapa verifique que el agregar nuevas componentes no rompa la funcionalidad previamente validada.

- Para el cálculo de la distancia euclideana, utilice en primera instancia el IP CORDIC disponible en el IP Catalog de Vivado.

- Una vez haya verificado la funcionalidad de su sistema, determine cual es la máxima frecuencia de reloj a la cual puede operar la lógica del processing core en su coprocesador. Para esto, siga un enfoque de prueba y error, modificando la frecuencia del reloj de salida del Clocking Wizard para encontrar la mayor frecuencia a la cual puede operar el sistema sin presentar fallas de timing estático, manteniendo las opciones de Vivado por defecto para síntesis lógica e implementación. Identifique, reporte, y analice el critical path. Para esta parte, puede simplificar el análisis omitiendo la lógica requerida para comunicación UART, y analizando solamente los paths en el dominio de reloj del `processing core`.

- En base al diagrama de módulos e interconexiones y a la frecuencia determinada en el paso anterior, determine una expresión analítica para la latencia de cada una de las operaciones. Para este análisis, asuma que los datos ya se encuentran guardados en memoria y **considere solamente el procesamiento/cálculo realizado en el dominio del** `processing core`. Para esta caracterización, considere como inicio el instante en que el Processing Core acepta la señal de inicio de una operación, con los operandos ya disponibles en memoria, y como término el ciclo en que se indica como válido el último resultado generado. No considere las latencias asociadas a UART ni a la serialización/deserialización.

- Utilice el analizador lógico integrado (ILA) de Vivado para depurar su diseño, observar el estado y flujo de las señales internas de la FPGA, y medir la latencia para cada una de las operaciones, considerando el escenario descrito en el punto anterior (solo lo asociado al processing core). Compare la latencia medida con la derivada de la expresión analítica del paso previo. En caso de existir diferencias, ajuste la expresión considerando todos los factores involucrados en la latencia. **El core ILA utiliza lógica y agrega overhead en la utilización de recursos y análisis de timing de su sistema, por lo que solo debe utilizarse para depuración y removerse de la implementación final una vez realizado su análisis.**

- Investigue sobre formas alternativas para calcular la raíz cuadrada, elija una, impleméntela, e intégrela a su sistema en reemplazo del IP CORDIC. Es válido utilizar otros IPs o códigos de terceros, citando debidamente las fuentes. Luego de verificar la equivalencia funcional, realice un análisis y prepare un cuadro comparativo para identificar los beneficios y desventajas de

las distintas formas de calcular la raíz cuadrada en hardware, considerando uso de recursos, precisión numérica, timing, complejidad de uso, etc.

Tome nota de todas sus observaciones, puntos que le hayan llamado la atención, y lecciones aprendidas durante el proceso, para luego reportarlas mediante una sección de discusión en el informe escrito.

Cualquier funcionalidad o característica que considere necesaria y no esté explícitamente especificada en el enunciado queda abierta a su decisión de diseño, lo cual debe ser reportado y justificado en su informe.

4. Indicaciones de entrega

La evaluación considerará la verificación funcional del sistema implementado a partir de los archivos fuente de su diseño y el documento del reporte escrito.

Para la parte práctica, debe entregarse una carpeta que incluya **solo los archivos fuente necesarios para implementar su diseño.** Esto incluye los archivos HDL (típicamente carpeta sources de su proyecto), constraints (típicamente carpeta constraints de su proyecto), y cualquier bloque de configuración de IP que haya instanciado. Considere solo la entrega de la versión final, incluyendo el uso de lógica alternativa al CORDIC para calcular la raíz cuadrada. No incluya el archivo de programación .bit, ya que este se generará al momento de la revisión y se probará en la tarjeta NexysA7 del profesor. Revise muy bien que está enviando los archivos correctos y suficientes para implementar su sistema. Se penalizará la nota si no se incluyen todos los archivos necesarios y también si incluye archivos innecesarios derivados del proceso de síntesis e implementación en su computador personal. Recuerde incluir el script de MATLAB con la interfaz de usuario que permite validar su diseño, con todas las indicaciones necesarias para hacerlo funcionar. El diseño entregado no debe incluir lógica asociada al ILA.

La implementación física debe contar con un grado mínimo de funcionalidad, partiendo por ser capaz de realizar la correcta escritura y lectura de datos en las memorias. En este contexto, se enfatiza la necesidad de que trabajen en forma incremental, agregando funcionalidad al sistema en base a complejidad creciente. Si no logra implementar correctamente alguna operación o funcionalidad específica, asegúrese de que las otras partes sí funcionen para optar a recibir puntaje parcial. Sea riguroso/a con las pruebas realizadas. Un sistema que funciona "*a veces*" se considerará como completamente incorrecto o no implementado.

El reporte escrito debe estructurarse como un reporte técnico, documentando las características de su diseño, los resultados, y sus observaciones, articulándolos para proveer un flujo narrativo coherente. El informe debe incluir al menos los aspectos señalados anteriormente (diagrama de bloques de alto nivel con las señales principales entre bloques, reportes y análisis cuantitativos sobre el uso de recursos lógicos y la caracterización de latencia, etc.) y cualquier otra información o análisis adicional que considere relevante incluir. Incluya también en el informe los diagramas de estado principales en caso de que haya usado FSMs. Los módulos y señales principales deben coincidir con los entregados en los archivos de diseño [6]. No incluya código en el informe a menos

[6] Una actividad inherente acá es identificar cuales son las señales relevantes que aportan al análisis y discusión. No es necesario describir todos y cada uno de los módulos y señales. Es un reporte de análisis, no un datasheet de su sistema

que lo considere necesario para ilustrar algún punto relevante. Recuerde incluir cualquier indicación especial que el usuario deba saber para hacer funcionar el device. Incluya también una sección de discusión de cierre sobre aspectos que resultaron interesantes, lecciones aprendidas, sugerencias para mejorar su diseño, etc.

Recuerde que se permite, y recomienda, el uso responsable de herramientas de Large Language Models (LLMs) para mejorar la presentación de su informe escrito.

5. Bitácora de trabajo

Para esta tarea, se solicita que registren el tiempo invertido en el proceso de desarrollo de las actividades en una bitácora como la mostrada en la Tabla 1. Esto considera actividades como búsqueda de información, análisis y discusión sobre aspectos de diseño, descripción de código, simulaciones, intentos fallidos, iteraciones del diseño, etc.

El uso de la bitácora tiene el propósito de ayudarlos a planificar su trabajo en sesiones y que puedan obtener una referencia para que puedan dimensionar su trabajo a futuro, además de proveer un insumo para análisis de carga del curso. Complete la bitácora de la forma más honesta y detallada posible, considerando el ***tiempo de trabajo efectivo*** aproximado en horas. Tiempo de análisis e iteraciones sobre el trabajo sin obtener resultados esperados si cuenta como tiempo efectivo. Tiempos de ocio y distracciones en el medio de sesiones de trabajo, si bien son necesarias, no cuentan como tiempo efectivo de trabajo.

Una vez completada la bitácora, indique el total de horas invertidas y presente una breve discusión sobre cuales fueron las partes más complicadas y las que tomaron más tiempo, e identifique si las dificultades asociadas a errores propios o aspectos intrínsicas del proceso de diseño. Reporte también lecciones aprendidas que podrían ayudarle a trabajar más efectivamente a futuro, sugerencias sobre que se podría hacer en el curso o que tópicos se podrían cubrir de antemano para ayudar en estos procesos, etc.

Es relevante que hagan una cuantificación honesta y objetiva. El número total de horas reportadas no tendrá efecto en la nota. Sí se evaluará la completitud de la bitácora y la calidad de la discusión realizada a partir de ella.

6. Actividades propuestas (no evaluadas pero recomendadas)

6.1. Exploración de múltiples runs de implementación

Vivado permite crear y ejecutar múltiples runs de síntesis e implementación para un mismo diseño, utilizando diferentes estrategias y configuraciones. Esta funcionalidad permite explorar distintas alternativas de implementación para un mismo sistema sin modificar el código HDL y comparar el impacto de las decisiones de la herramienta sobre la calidad de los resultados.

Utilizando como base el diseño final desarrollado para esta tarea, cree al menos tres implementation runs que utilicen el mismo resultado de síntesis y las mismas restricciones temporales, pero distintas estrategias de implementación disponibles en Vivado. Incluya como referencia la estrategia por defecto y seleccione al menos dos estrategias adicionales orientadas, por ejemplo, a mejorar

Tabla 1: Ejemplo de tabla bitácora (formato propuesto, puede utilizar otro formato).

Día	**Hora**		**Actividades**
	Inicio	**Término**	
30 de septiembre	17:00	21:00	Búsqueda de información, estudio y discusión sobre aritmética binaria. Revisión de ejemplos.
1 de octubre	10:00	12:00	Preparación y revisión de diagramas de alto nivel para módulos principales.
1 de octubre	16:00	19:00	Codificación HDL y simulación de algunos módulos. Tuvimos problemas con el funcionamiento de Vivado y varios errores de síntesis. Corregimos los errores de síntesis, pero ahora las simulaciones no tienen sentido. Vivado está malo así que procedemos a instalarlo de nuevo.
2 de octubre	17:00	19:00	Luego de reinstalar Vivado seguimos con los mismos problemas. Al observar mejor, nos damos cuenta de que era un problema de diseño y descripción, así que repetimos el proceso poniendo más atención a los detalles.

desempeño, reducir área o explorar alternativas de ubicación y ruteo.

Ejecute los runs y compare los resultados obtenidos. Para cada caso, registre al menos:

- Estrategia utilizada
- Worst Negative Slack (WNS);
- Utilización de LUTs, FFs, BRAMs y DSPs;
- Características del critical path;
- Tiempo aproximado requerido (runtime) para realizar la implementación.

Organice los resultados en una tabla y analice las diferencias observadas. Determine si las distintas estrategias producen diferencias relevantes y discuta los compromisos entre calidad de resultados, uso de recursos y tiempo requerido por las herramientas.

Se recomienda además explorar opciones para correr varios runs en paralelo, configurando el parámetro *Number of jobs* disponible en la ventana Launch Runs. Si los recursos de su computador lo permiten, configure más de un job para ejecutar varios runs en paralelo. Observe la diferencia en el tiempo total requerido y considere que cada run corresponde a un proceso independiente que consume recursos de CPU y memoria.

El objetivo de esta actividad no es encontrar una configuración universalmente óptima, sino familiarizarse con el concepto de exploración del espacio de diseño (*Design Space Exploration*) y observar que un mismo diseño RTL puede producir implementaciones físicas diferentes dependiendo de las estrategias, restricciones y objetivos utilizados por las herramientas.

6.2. Estudio adicional

Hay diversas opciones avanzadas de Vivado para explorar alternativas de síntesis e implementación y mejorar la calidad de los resultados (*Quality of Results*, QoR). Para profundizar en estos aspectos, se recomienda revisar la documentación oficial de AMD, en particular *Vivado Design Suite User Guide: Design Flows Overview* (UG892). *Vivado Design Suite User Guide: Synthesis* (UG901), *Vivado Design Suite User Guide: Implementation* (UG904) y *Vivado Design Suite User Guide: Design Analysis and Closure Techniques* (UG906). Explore en estas referencias conceptos asociados a *Project Mode* y *Non-Project Mode*, estrategias y directivas de síntesis e implementación, automatización mediante TCL y técnicas para análisis y optimización de la calidad de los resultados.